

Phần 6 - Distributed Architecture Styles

Mục lục

6.1 Tổng quan Phần 6: Distributed Architecture Styles	3
Nếu bạn đến từ Data Science	4
Distributed style ladder	4
Bài trong phần	4
6.2: Service-based Architecture	4
6.3: Microservices	4
6.4: Event-Driven Architecture	4
6.5: Space-Based Architecture	4
Quy tắc chọn style	5
Câu hỏi quyết định	5
Default	5
Common challenges của distributed	5
Trade-off summary table	5
6.2 Service-based Architecture	6
Topology	6
Nếu bạn đến từ Data Science	6
So với Monolith	7
So với Microservices	7
Khi dùng Service-based	7
Implementation patterns	7
Pattern 1: Shared database, separate schema	7
Pattern 2: Database per domain (group of related services)	8
Pattern 3: Database per service (gần microservices)	8
Inter-service communication	8
Sync HTTP/REST	8
gRPC	9
Message queue (async)	9
Trade-off với QA	9
Migration path	9
Bước 1: Modularize monolith (Bài 3.4)	10
Bước 2: Extract first service	10
Bước 3: Setup infrastructure	10
Bước 4: Tách tiếp theo theo bounded context	10
Bước 5: Stay đây, hoặc tiến lên microservices	10
Sai lầm thường gặp	10
Sai lầm 1: Tách service quá nhỏ	10

Sai lầm 2: Tách service theo layer (horizontal slicing)	10
Sai lầm 3: Shared DB không có schema discipline	10
Sai lầm 4: Không có observability từ ngày 1	10
Sai lầm 5: Quên circuit breaker	11
Real-world examples	11
Tóm tắt	11
6.3 Microservices	12
Nếu bạn đến từ Data Science	12
Topology	12
SRP scale lên service level	13
Khi thực sự cần	13
Điều kiện 1: Team scale lớn	13
Điều kiện 2: Complex domain với nhiều bounded context	13
Điều kiện 3: Independent scaling per capability	13
Điều kiện 4: DevOps mature	13
Điều kiện 5: Business case justify cost	13
Cost operations	13
1. Infrastructure	14
2. Development overhead	14
3. Operational complexity	14
4. Testing complexity	14
5. Debugging	14
Bounded Context = Service boundary	14
Heuristic identify bounded context	15
Ví dụ e-commerce	15
Distributed Transaction: Saga Pattern	15
Choreography saga	15
Orchestration saga	16
Eventual Consistency	16
Trade-off với QA	16
Migration path	17
Sai lầm thường gặp	17
Sai lầm 1: Premature microservices	17
Sai lầm 2: Distributed monolith	17
Sai lầm 3: Shared database	17
Sai lầm 4: Sync everything	17
Sai lầm 5: Ignore observability	17
Sai lầm 6: One developer per service	17
Tóm tắt	17
6.4 Event-Driven Architecture	18
Khái niệm	18
Nếu bạn đến từ Data Science	18
Sync vs Async	19
Hai topology	19
Mediator (Orchestration)	19
Broker (Choreography)	20
Event Bus (Message Broker)	20

CAP Theorem	21
CP systems	21
AP systems	21
EDA thường là AP	21
Event design	21
Event naming	21
Event content	22
Schema evolution	22
Patterns trong EDA	22
Event Sourcing	22
CQRS (Command Query Responsibility Segregation)	23
Saga (đã nhắc Bài 6.3)	23
Trade-off với QA	23
Use cases mạnh	24
Anti-patterns	24
Anti-pattern 1: Event = remote procedure call	24
Anti-pattern 2: Tightly-coupled events	24
Anti-pattern 3: No ordering guarantee	24
Anti-pattern 4: No replay capability	24
Anti-pattern 5: At-least-once delivery, idempotency missing	24
Tóm tắt	24
6.5 Space-Based Architecture	26
Nếu bạn đến từ Data Science	26
Vấn đề mà Space-Based giải	26
Topology	26
Tên gọi “space-based”	26
Cách hoạt động	26
Đọc	26
Ghi	26
Scale	26
Failover	28
Khi dùng	28
Trade-off	28
Vendors	28
Variant: Distributed Cache + DB	28
Sai lầm thường gặp	29
Sai lầm 1: Choose SBA cho load bình thường	29
Sai lầm 2: Quên data persistence	29
Sai lầm 3: Strong consistency expectation	29
Sai lầm 4: Complex query	29
Tóm tắt	29
Tổng kết Phần 6	29

6.1 Tổng quan Phần 6: Distributed Architecture Styles

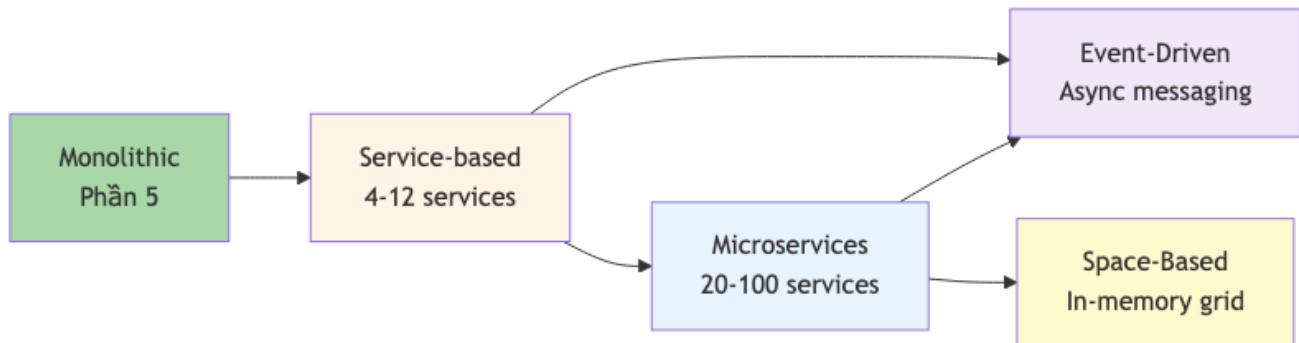
Tóm tắt một dòng: Phần này dạy 4 style phân tán phổ biến nhất, từ “nhẹ” (Service-based) tới “phức tạp” (Space-Based). Mỗi style giải quyết một bài toán scale + decoupling cụ thể, với cost distributed khác nhau.

Nếu bạn đến từ Data Science

Distributed styles hấp dẫn vì nghe giống production-ready, nhưng cũng là nơi DS team dễ over-engineer nhất. Một ML platform nhỏ không cần bắt đầu bằng Kubernetes, Kafka, 30 microservices và service mesh. Hãy hỏi Quality Attributes trước: cần freshness theo giây hay theo ngày? Load bao nhiêu? Team có người trực production không? Có cần independent deploy từng component không?

Thông thường, path an toàn là: modular package hoặc modular monolith, sau đó service-based, sau đó mới microservices/event-driven cho phần thật sự cần scale hoặc decoupling.

Distributed style ladder



Hình 1: Sơ đồ 22

Đi từ trái sang phải = complexity tăng + capability tăng. Hầu hết hệ stop ở Service-based hoặc Microservices. Event-Driven thường overlay lên Service-based/Microservices, không thay thế.

Space-Based là special case cho extreme load.

Bài trong phần

6.2: Service-based Architecture

“Macro-services”. 4-12 coarse services + shared database (hoặc shared DB per domain). Middle ground giữa monolith và microservices. Pragmatic cho most enterprise.

6.3: Microservices

20-100 fine-grained services + database per service. Hype mạnh 2015-2020. Đầu tư operational lớn. Phù hợp cho large team (50+ engineers) + complex domain.

6.4: Event-Driven Architecture

Async messaging. Producer emit events, consumer subscribe. Decoupling cực mạnh. Hai topology: Broker (peer-to-peer) và Mediator (orchestration). CAP theorem relevant.

6.5: Space-Based Architecture

In-memory data grid + async DB sync. Designed cho extreme load (Black Friday, ticket sales). Cassandra/Hazelcast/Apache Ignite. Niche nhưng powerful.

Quy tắc chọn style

Câu hỏi quyết định

1. **Team size:** < 15 □ monolith hoặc service-based. 15-50 □ service-based. 50+ □ microservices.
2. **Domain complexity:** Đơn giản □ monolith. Trung □ service-based. Phức tạp + nhiều bounded context □ microservices.
3. **Scaling requirements:** Uniform □ monolith. Per-domain □ service-based / microservices.
4. **Async needs:** Many “fire-and-forget” workflows □ Event-driven.
5. **Extreme load:** > 1M concurrent users với strict latency □ Space-based.

Default

Most project: **Service-based**. Combine với event-driven cho async parts. Avoid full microservices unless thật sự cần.

Common challenges của distributed

Cứ là distributed, gặp:

- **Network failure:** 8 fallacies (Bài 5.2).
- **Distributed transaction:** Saga pattern thay 2PC.
- **Consistency:** eventual thay strong (CAP).
- **Distributed tracing:** Jaeger/Zipkin.
- **Service discovery:** Consul, Eureka, Kubernetes Service.
- **Configuration:** centralized config server.
- **Secret management:** Vault, AWS Secrets Manager.
- **Auth & authorization:** JWT, mTLS, OAuth.

Mỗi challenge cần investment. Đây là cost của distributed.

Tools mature: Spring Cloud (Java), Kubernetes ecosystem (cloud-agnostic), AWS App Runner / GCP Cloud Run (managed).

Trade-off summary table

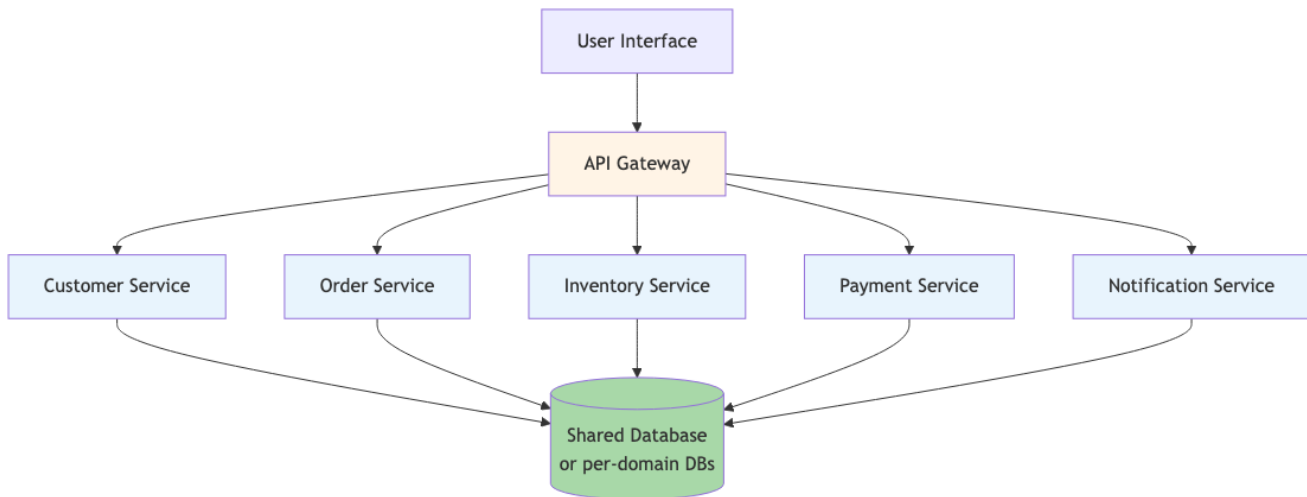
Aspect	Service-based	Microservices	Event-Driven	Space-Based
Service count	4-12	20-100	Varies	Varies
DB	Shared/per-domain	Per service	Varies	In-memory grid + async
Communication	Sync (HTTP/gRPC)	Sync + async	Async events	Memory grid
Consistency	Strong (shared DB)	Eventual	Eventual	Eventual (lazy sync)
Operational cost	Medium	High	High	Very high
Best for	Enterprise medium	Large scale, complex domain	Decoupling, real-time	Extreme load

Bài tiếp: [Service-based Architecture](#), pragmatic middle ground.

6.2 Service-based Architecture

Tóm tắt một dòng: Tách hệ thành 4-12 coarse-grained services chia sẻ database (hoặc shared per-domain). Nhận được lợi ích team independence + scalability per-service nhưng không gặp full cost của microservices. Là “default distributed” cho most enterprise.

Topology



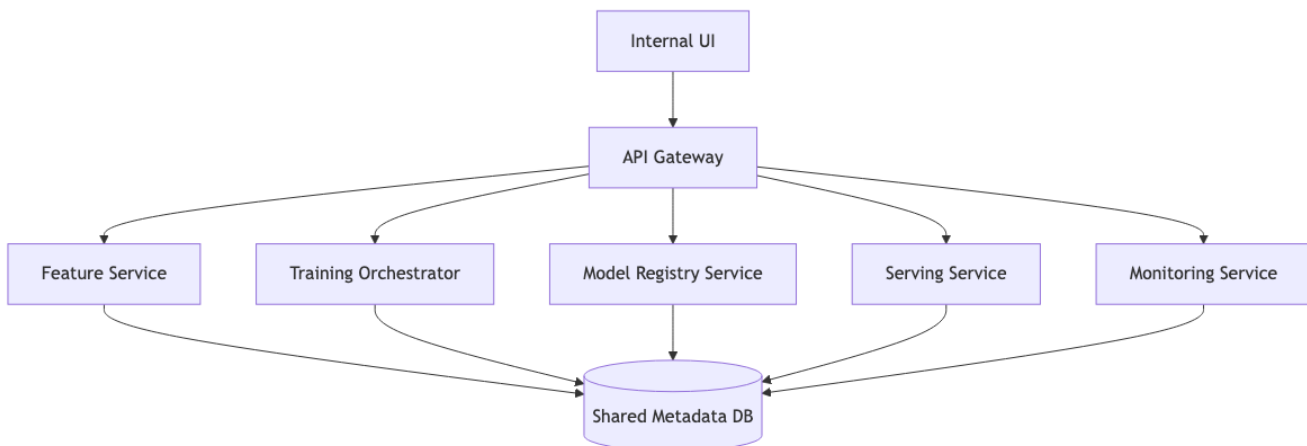
Hình 2: Sơ đồ 23

Đặc điểm:

- **4-12 coarse services:** mỗi service own một domain (Customer, Order, ...) ở mức coarse-grained.
- **Database shared** (hoặc per-domain): không “database per service” như microservices.
- **Sync communication:** HTTP/gRPC giữa services. Có thể overlay async với event.
- **API Gateway** ở front: routing, auth, rate limiting.

Nếu bạn đến từ Data Science

Service-based thường là điểm bắt đầu thực tế nhất cho ML platform vừa và nhỏ. Bạn không cần 50 microservices để productionize model. Thường 4-8 service coarse-grained là đủ:



Hình 3: Sơ đồ 24

Trong topology này, team vẫn có boundary rõ: feature team own Feature Service, platform team own Registry/Serving, data team own Training Orchestrator. Nhưng operational cost vẫn thấp hơn full microservices vì database và deployment model chưa bị tách quá mịn.

Nếu team DS/ML của bạn nhỏ hơn 15 người, hãy rất cẩn thận trước khi chọn microservices. Service-based hoặc modular monolith thường là bước trưởng thành hơn.

So với Monolith

- Service deploy độc lập (team velocity).
- Mỗi service scale riêng theo load.
- Fault isolation từng phần.
- Tech stack có thể khác (Java cho Customer, Go cho Payment).

So với Microservices

Aspect	Service-based	Microservices
Số service	4-12	20-100+
Granularity	Coarse (1 service = 1 bounded context, ~5-10 entity)	Fine (1 service = 1 entity hoặc 1 capability)
Database	Shared / per-domain	Per service strictly
Communication	Sync (HTTP/gRPC)	Sync + async heavily
Transactions	Có thể dùng DB transaction cross-service nếu shared	Bắt buộc saga / eventual
Operational cost	Medium	High
Time to first deploy	1-3 tháng	6-12 tháng

Service-based là “microservices lite”, get majority of benefits, pay minority of cost.

Khi dùng Service-based

☐ Phù hợp:

- Enterprise medium: 15-50 engineers.
- Domain có 4-12 bounded context rõ ràng.
- Cần team independence ở deploy nhưng chưa cần full isolation.
- Đã try monolith và pain với merge conflict / deploy bottleneck.
- Không có team SRE/DevOps mature để operate microservices.

☐ Không phù hợp:

- Team < 10 (overkill, dùng monolith).
- Team > 100 với complex domain (chưa đủ, dùng microservices).
- Cần extreme independence (vd: regulated industry với separate compliance scope per service).

Implementation patterns

Pattern 1: Shared database, separate schema

Tất cả service connect cùng DB instance, mỗi service own schema riêng.

```
-- Customer service owns
CREATE SCHEMA customer_svc;
CREATE TABLE customer_svc.customers (...);
```

```
CREATE TABLE customer_svc.addresses (...);

-- Order service owns
CREATE SCHEMA order_svc;
CREATE TABLE order_svc.orders (...);
```

Pros: low ops cost (1 DB), can JOIN cross-schema if needed.

Cons: DB là single point of failure, schema migration coordination.

Pattern 2: Database per domain (group of related services)

Group services theo domain, mỗi group có DB riêng.

Customer domain:

- Customer service
- Address service
- Database CustomerDB

Order domain:

- Order service
- Inventory service
- Pricing service
- Database OrderDB

Payment domain:

- Payment service
- Refund service
- Database PaymentDB

Pros: isolation hơn pattern 1, fault tolerance tốt hơn.

Cons: cross-domain query phức tạp (cần API call hoặc data replication).

Pattern 3: Database per service (gần microservices)

Mỗi service có DB riêng. Khác microservices: vẫn allow sync HTTP call cross-service, không *strict* event-driven.

Pros: full data isolation.

Cons: distributed transaction complexity = microservices.

Inter-service communication

Sync HTTP/REST

Common nhất. Service A gọi service B qua HTTP.

```
# In Order Service
response = requests.get(
    f"{CUSTOMER_SERVICE_URL}/customers/{customer_id}",
    headers={"X-Trace-Id": trace_id},
```



```

    timeout=2.0
)
customer = response.json()

```

Cần:

- **Timeout:** tránh hang khi service B chậm.
- **Retry với backoff:** handle transient failure.
- **Circuit breaker:** khi B down, ngừng gọi trong N giây (Hystrix, Resilience4j).
- **Fallback:** nếu B down, có default behavior?

gRPC

Tốt cho high-throughput internal communication. Protobuf strict contract.

```

service CustomerService {
  rpc GetCustomer(GetCustomerRequest) returns (Customer);
}

```

Pros: 2-5x faster than REST/JSON, strict typing.

Cons: harder to debug (binary protocol), không native browser-friendly.

Message queue (async)

Service A enqueue message, service B consume later. Decoupling tốt.

Vd: Order Service push “order.placed” event, Notification Service consume.

Sẽ đi sâu ở Bài 6.4 (Event-Driven).

Trade-off với QA

QA	Service-based	Monolith ref	Microservices ref
Simplicity	□□□	□□□□□	□
Operational cost	□□□	□□□□□	□
Scalability	□□□□	□	□□□□□
Team independence	□□□□	□	□□□□□
Fault isolation	□□□	□	□□□□□
Performance (latency)	□□□	□□□□□	□□
Data consistency	□□□□ (shared DB)	□□□□□	□□ (eventual)
Time to first deploy	□□□	□□□□□	□
Testability	□□□	□□□□	□□

Insight: service-based “ổn” trên mọi chiều, không “xuất sắc” ở chiều nào nhưng cũng không “tệ” ở chiều nào. Pragmatic default.

Migration path

Từ monolith lên service-based:

Bước 1: Modularize monolith (Bài 3.4)

Tổ chức monolith thành strict modules. Build tool enforce import boundary.

Bước 2: Extract first service

Chọn module có pain rõ nhất (vd: scale separately needed) □ tách thành service. Dùng Strangler Fig (Bài 5.2).

Bước 3: Setup infrastructure

API gateway, service discovery, monitoring, distributed tracing. Setup *trước* khi tách service thứ 2.

Bước 4: Tách tiếp theo theo bounded context

Mỗi quarter tách 1-2 service. Trong 12-18 tháng có service-based architecture với 5-10 services.

Bước 5: Stay đây, hoặc tiến lên microservices

Khi nào team > 50 + có pain rõ với coarse services □ consider microservices. Đa số dừng ở service-based.

Sai lầm thường gặp**Sai lầm 1: Tách service quá nhỏ**

5 service cho domain Order: OrderCreator, OrderUpdater, OrderDeleter, OrderReader, OrderValidator. Quá granular cho service-based.

Fix: 1 OrderService duy nhất ở mức coarse. CRUD nội bộ một service.

Sai lầm 2: Tách service theo layer (horizontal slicing)

UserUIService, UserBusinessService, UserDataService. Distributed monolith.

Fix: tách theo domain (vertical slicing). User là 1 service end-to-end.

Sai lầm 3: Shared DB không có schema discipline

Mọi service đọc/ghi mọi table. Schema migration phá service khác.

Fix: schema-per-service. Strict ownership. Schema thay đổi via service interface, không direct SQL.

Sai lầm 4: Không có observability từ ngày 1

Tách service mà không có distributed tracing, centralized logging. Debug = mò mẫm.

Fix: Jaeger + ELK stack từ tuần 1. Trace ID propagate qua mọi service call.

Sai lầm 5: Quên circuit breaker

Service A gọi B. B down $\square$ A hang $\square$ A's clients timeout $\square$ cascade failure.

Fix: circuit breaker mọi inter-service call. Library: Hystrix, Resilience4j, Polly.

Real-world examples

- **Shopify** (early stage 2010-2015): service-based với ~10 services trước khi migrate tiếp.
- **GitHub** (long time): service-based với Ruby + Go.
- **Stripe**: nhiều service-based regions, gradual migration to microservices for some domains.
- **Hầu hết enterprise truyền thống** (banking, insurance): service-based với mainframe + Java services overlay.

Tóm tắt

- **Service-based**: 4-12 coarse services, shared/per-domain DB.
- **Middle ground**: lợi ích team independence + scalability nhưng không cost full microservices.
- **Pragmatic default** cho most enterprise.
- **Migration path**: monolith $\square$ modular monolith $\square$ extract services gradually.
- **Tránh**: over-fragment, horizontal slicing, schema chaos, no observability, no circuit breaker.

Bài tiếp: [Microservices](#), next step khi service-based không đủ.

6.3 Microservices

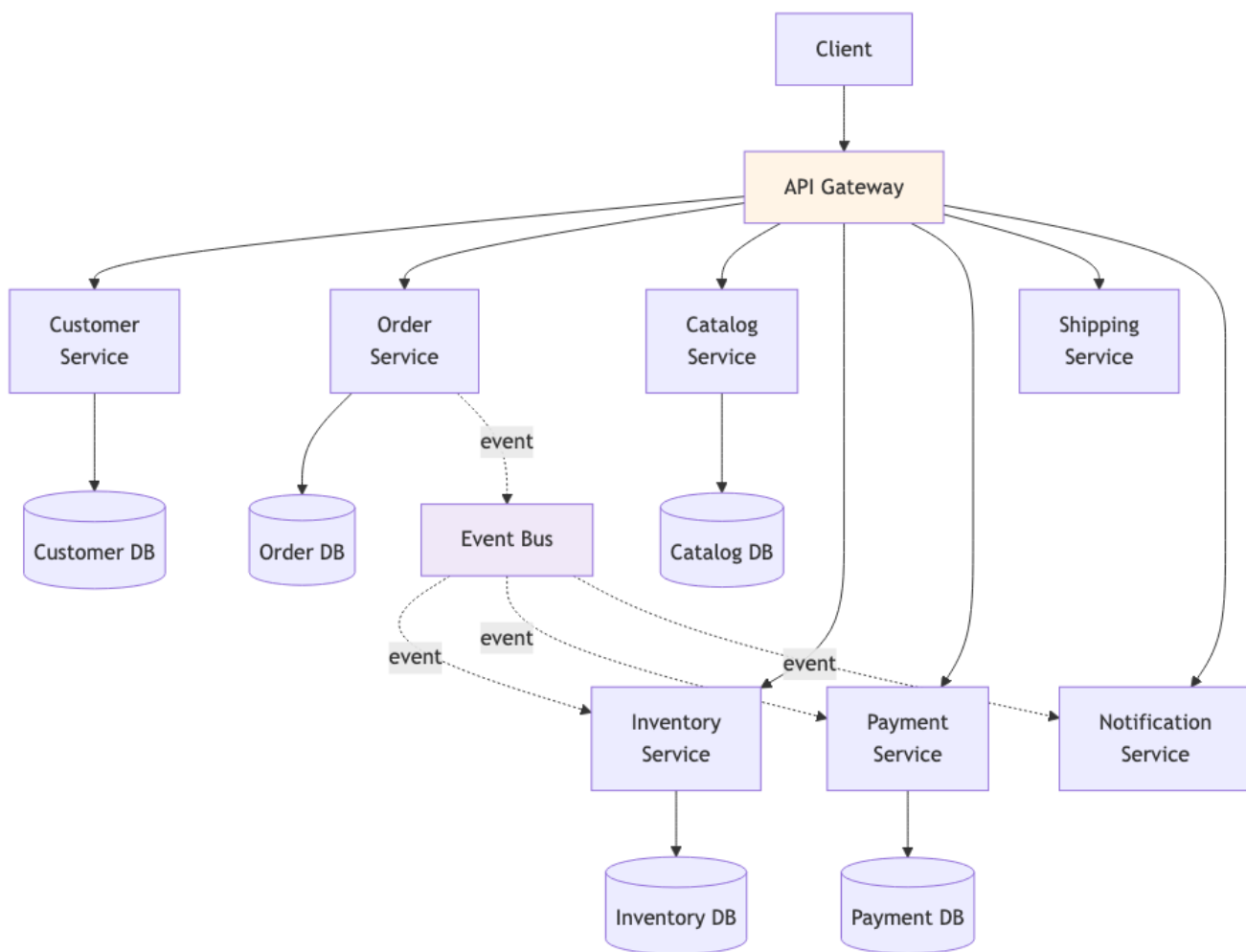
Tóm tắt một dòng: Tách hệ thành nhiều service nhỏ, mỗi service own một bounded context và database riêng, communication chủ yếu qua API + event. Lợi ích cực mạnh cho team scale + independent deploy, nhưng cost ops khổng lồ. Chỉ phù hợp khi team ≥ 50 và đã có DevOps mature.

Nếu bạn đến từ Data Science

Microservices không phải câu trả lời mặc định cho MLOps. Nếu team nhỏ, microservices có thể làm bạn mất nhiều thời gian vào deployment, networking, tracing, auth, versioning, monitoring hơn là cải thiện model. Một ML platform có 5 service coarse-grained thường thực tế hơn 30 service nhỏ.

Microservices chỉ đáng cân nhắc khi domain đủ lớn: feature platform có team riêng, serving platform có team riêng, training orchestration có team riêng, monitoring có team riêng, và mỗi team cần deploy/scale độc lập. Nếu vẫn cùng một nhóm 5 người sửa mọi service, bạn có thể chỉ đang tạo distributed monolith.

Topology



Hình 4: Sơ đồ 25

Đặc trưng:

- **Fine-grained:** 1 service = 1 bounded context (hoặc 1 capability).
- **Database per service:** strict isolation. Service A không touch DB của B.
- **API + event:** sync HTTP/gRPC cho query; async event cho integration.
- **Independent deploy:** mỗi service own pipeline.
- **Polyglot:** ngôn ngữ/tech stack có thể khác giữa services.

SRP scale lên service level

Bài 2.3 nói SRP: “mỗi module một actor”. Microservices áp dụng SRP ở service level:

- Mỗi service own một bounded context (ngôn ngữ chung, một team).
- Service thay đổi vì *một lý do* (business rule trong bounded context đó thay đổi).
- Service deploy độc lập = SRP đảm bảo.

Đây là benefit lớn nhất của microservices.

Khi thực sự cần

Đừng làm microservices “vì cool”. Cần:

Điều kiện 1: Team scale lớn

≥ 50 engineers, ≥ 5 sub-teams. Mỗi team own 2-5 service. Service-based không đủ team independence.

Điều kiện 2: Complex domain với nhiều bounded context

10+ bounded context rõ ràng (DDD). Service-based 4-12 service không cover đủ.

Điều kiện 3: Independent scaling per capability

Vd: search heavy (10x load), comment light. Microservices cho phép scale chỉ search.

Điều kiện 4: DevOps mature

Đã có:

- Container orchestration (Kubernetes hoặc managed equivalent).
- CI/CD per service.
- Service mesh (Istio/Linkerd hoặc lite alternative).
- Centralized logging (ELK/Loki).
- Distributed tracing (Jaeger/Zipkin).
- Monitoring + alerting (Prometheus + Grafana).
- On-call rotation + incident management.

Nếu thiếu ≥ 3 trong list trên, đừng microservices yet.

Điều kiện 5: Business case justify cost

Microservices tăng infrastructure cost 3-10x, dev cost 1.5-2x (initial). Phải justify bằng business value (faster time-to-market, larger scale).

Cost operations

Cost ẩn của microservices, mỗi cái phải có ngân sách:

1. Infrastructure

- Mỗi service: 2-3 instances cho HA = 100+ instances cho 50 services.
- Service mesh: thêm sidecar (1 sidecar per pod).
- Load balancer per service.
- Database per service (50+ databases).
- Message broker (Kafka cluster) cho async.

Cost: \$5,000-\$50,000+/month tùy scale, vs \$500/month cho monolith equivalent.

2. Development overhead

- Mỗi service: own repository, CI/CD, README, on-call runbook.
- Cross-service feature: coordinate 3-5 service changes, contract testing.
- Distributed transaction: saga pattern, compensation logic.

Cost: 1.5-2x dev time cho feature cross-service.

3. Operational complexity

- Distributed tracing setup.
- Centralized logging.
- Secret management (Vault).
- Service discovery.
- Authentication propagation (JWT/mTLS).

Cost: 1-2 SRE engineers dedicated minimum.

4. Testing complexity

- Unit test: dễ (per service).
- Integration test: medium (per service với mock).
- Contract test: medium-hard (Pact, Spring Cloud Contract).
- End-to-end test: hard (need spin up many services), flaky.

Cost: 2-3x test infrastructure vs monolith.

5. Debugging

- Bug cross-service: trace qua N services.
- Performance issue: profile mỗi service.
- Data inconsistency: investigate eventual consistency timing.

Cost: 2-5x debugging time per incident.

Bounded Context = Service boundary

Câu hỏi nóng: làm sao tách service đúng?

Trả lời: **map service lên bounded context (DDD).**

Heuristic identify bounded context

1. **Ubiquitous language**: trong context X, từ “Customer” có nghĩa cụ thể. Cross context, nghĩa khác $\square$ boundary.
2. **Team ownership**: 1 team thuộc 1 context. Conway’s law: team boundary = context boundary.
3. **Change frequency**: code thay đổi cùng nhau ở cùng context. Code rarely cùng change $\square$ cross context.
4. **Data ownership**: mỗi entity primary owner ở 1 context. Đọc/ghi từ context khác qua API.

Ví dụ e-commerce

- **Catalog**: Products, Categories. Owned by Product team.
- **Order**: Orders, OrderItems. Owned by Order team.
- **Customer**: Profiles, Addresses. Owned by Customer team.
- **Inventory**: Stock, Reservations. Owned by Inventory team.
- **Payment**: Transactions, Refunds. Owned by Payment team.
- **Shipping**: Shipments, Tracking. Owned by Logistics team.

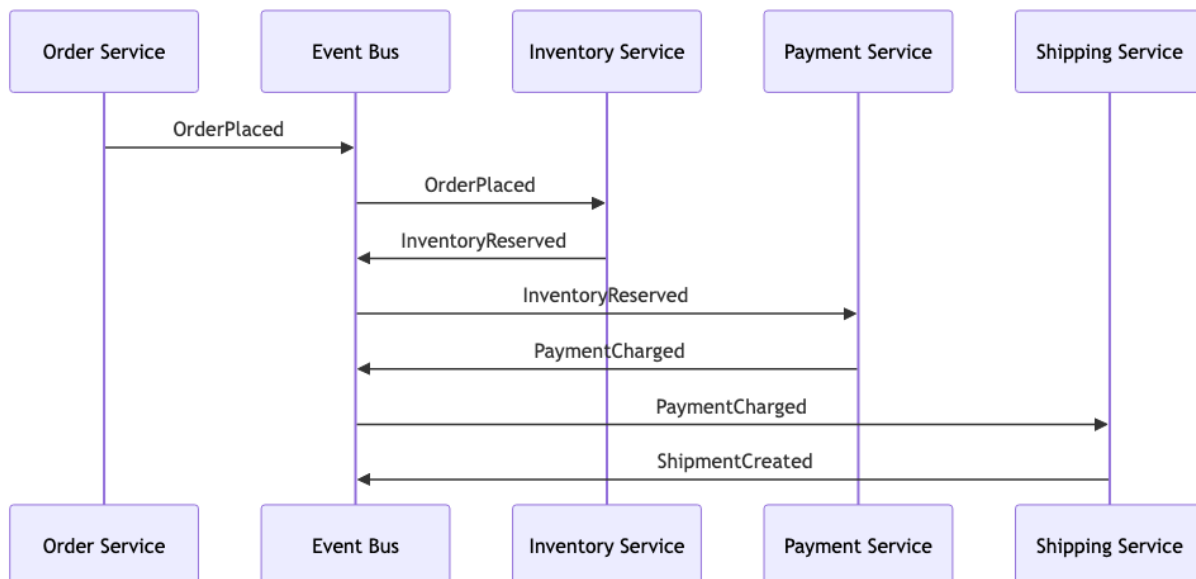
6 bounded contexts $\square$ 6+ services (mỗi context có thể có sub-service).

Distributed Transaction: Saga Pattern

Microservices không thể dùng 2PC (two-phase commit). Pattern thay: **Saga**.

Choreography saga

Mỗi service emit event sau action. Service khác react.



Hình 5: Sơ đồ 26

Pros: loose coupling, no central orchestrator.

Cons: hard to visualize flow, debugging khó.

Orchestration saga

Central orchestrator điều phối flow.

```
class PlaceOrderSaga:
    def execute(self, order):
        try:
            reservation = inventory_svc.reserve(order)
            payment = payment_svc.charge(order)
            shipment = shipping_svc.create(order)
        except Exception:
            # Compensation
            if shipment: shipping_svc.cancel(shipment)
            if payment: payment_svc.refund(payment)
            if reservation: inventory_svc.release(reservation)
            raise
```

Pros: flow rõ ràng, easier debug.

Cons: orchestrator là single point of complexity.

Choose tùy team prefer. Choreography phổ biến hơn trong microservices community.

Eventual Consistency

Microservices = eventual consistency (most case). Implications:

- “Inventory shows 5, but you ordered, then someone else ordered too”, both succeed, after 100ms one is reverted.
- User experience phải handle stale data (“processing...”, optimistic UI).
- Business team phải hiểu data có thể stale 100ms-30s.

Strong consistency khả thi cho 1 bounded context (within service DB) nhưng cross-service thường eventual.

Trade-off với QA

QA	Microservices
Scalability per service	□□□□□
Team independence	□□□□□
Deployability	□□□□□
Fault isolation	□□□□□
Polyglot tech stack	□□□□□
Operational complexity	□
Simplicity	□
Distributed transaction	□□ (saga complexity)
Latency	□□ (network overhead)
Testing	□□
Initial cost	□
Long-term agility (large team)	□□□□□

Tối ưu cho large team + scale. Sacrificing simplicity + cost.

Migration path

Từ monolith hoặc service-based lên microservices:

1. **Modular monolith** trước (Bài 5.2).
2. **Extract first 1-2 services** (Strangler Fig).
3. **Setup full DevOps stack** trước khi extract thứ 3.
4. **Tách dần theo bounded context**.
5. **Monitor cost vs benefit** ở mỗi milestone. Reverse course nếu cost > benefit.

Tránh big-bang rewrite. Risk cao, fail rate ~70%.

Sai lầm thường gặp

Sai lầm 1: Premature microservices

Startup 10 người làm 30 services. Operational hell. Pivot lại to modular monolith trong 6 tháng.

Sai lầm 2: Distributed monolith

Đã nói (Bài 3.4). Microservices tách sai = distributed monolith.

Sai lầm 3: Shared database

“Tạm thời” share. Sau 1 năm, schema migration coordinate 10 service. Hết lợi ích isolation.

Sai lầm 4: Sync everything

Mọi inter-service call sync HTTP. Latency cascade. Bug cross-service blocking.

Fix: dùng async event cho integration. Sync chỉ cho user-facing query.

Sai lầm 5: Ignore observability

Tách 30 service mà không có distributed tracing. Mỗi bug = 4 giờ debugging.

Fix: distributed tracing từ ngày 1. Correlation ID propagate qua mọi call.

Sai lầm 6: One developer per service

“Mỗi developer own 5 service” □ bus factor = 1. Khi dev nghỉ, service không ai maintain.

Fix: 2-pizza team (5-8 người) own 2-5 service. Knowledge shared.

Tóm tắt

- **Microservices**: 20+ fine-grained services, DB per service, async event-driven heavily.
- **SRP scale lên service**.
- **Cần điều kiện**: large team, complex domain, DevOps mature, business justify.
- **Cost ops khổng lồ**: infrastructure, dev, ops, testing, debugging.
- **Saga pattern** cho distributed transaction.
- **Eventual consistency** là norm.
- **Trade-off**: extreme team independence + scalability, sacrificing simplicity + cost.

Bài tiếp: [Event-Driven Architecture](#), async pattern overlay.

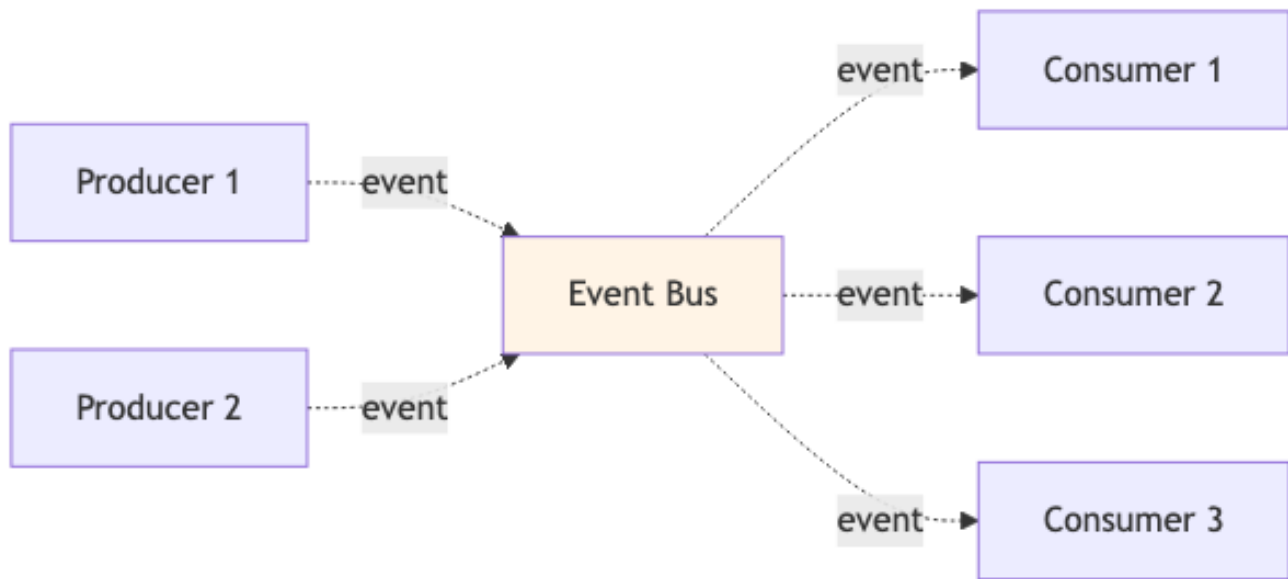
6.4 Event-Driven Architecture

Tóm tắt một dòng: Hệ giao tiếp qua event async thay vì call sync. Producer không biết ai consume. Decoupling cực mạnh, throughput cao, scalable. Cost - eventual consistency, debugging hard, message ordering.

Khái niệm

Event = “something happened in the past”. Vd: OrderPlaced, PaymentReceived, UserSignedUp.

Event-Driven Architecture (EDA) = hệ trong đó hầu hết communication qua event async, không phải request/response sync.



Hình 6: Sơ đồ 27

Producer không biết ai consume. Consumer subscribe topic/queue. Decoupling at runtime (knowing who exists).

Nếu bạn đến từ Data Science

Event-Driven Architecture rất gần với streaming data/ML. Một event không phải là một request hỏi đáp ngay lập tức, mà là một sự kiện đã xảy ra: TransactionCreated, UserClickedProduct, FeatureUpdated, ModelRegistered, PredictionMade.

Trong fraud detection, transaction event có thể đi qua nhiều consumer song song:

- Feature aggregator cập nhật online features.
- Fraud model scorer tính risk score.
- Audit logger lưu event để điều tra sau này.
- Monitoring service đo drift và error rate.
- Notification service gửi alert nếu score vượt threshold.

Producer của transaction không cần biết tất cả consumer này. Nó chỉ publish event. Đây là lợi ích lớn nhất của EDA: thêm consumer mới không bắt producer sửa code. Nhưng cái giá là debugging khó hơn, consistency thường là eventual, và bạn phải nghĩ nghiêm túc về duplicate, ordering, schema evolution.

Sync vs Async

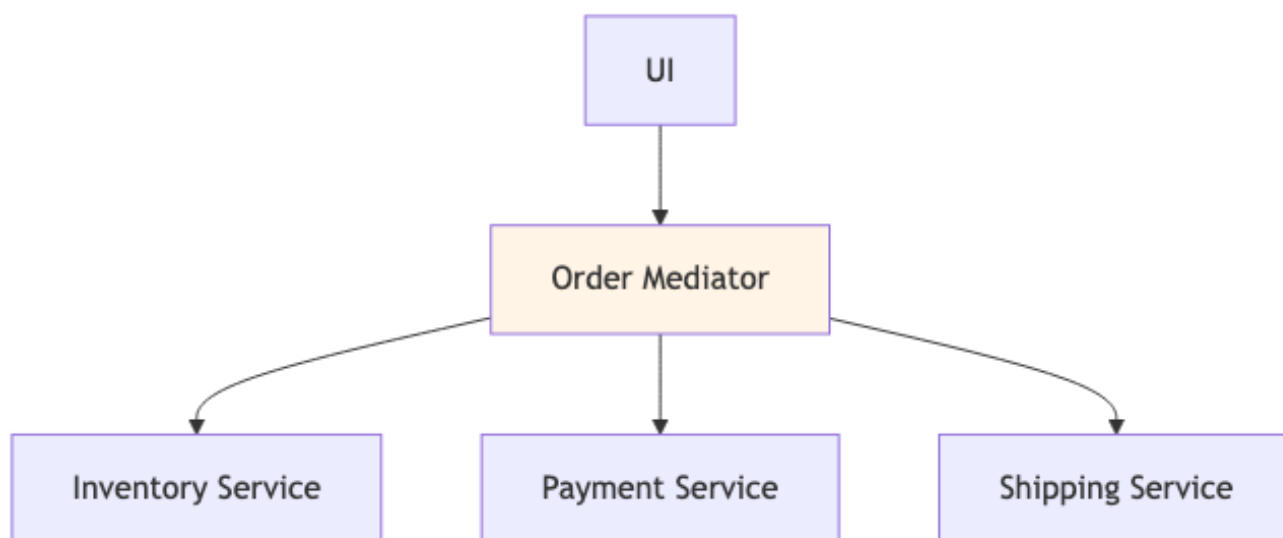
Aspect	Sync (request/response)	Async (event-driven)
Coupling	Producer biết consumer	Producer không biết
Timing	Wait for response	Fire and forget
Failure mode	Caller handle immediately	Retry/dead-letter queue
Latency	Sum of all calls	Producer latency only
Throughput	Limited (sync wait)	Cao (parallel async)
Use case	Read query, immediate need	Trigger action, integration

EDA tốt cho integration; sync vẫn tốt cho user-facing query.

Hai topology

Mediator (Orchestration)

Central mediator điều phối flow:



Hình 7: Sơ đồ 28

Mediator có logic: “step 1, step 2, step 3, rollback if X”.

Pros:

- Flow visible (đọc mediator code biết flow).
- Easy add new step.
- Compensation logic centralized.

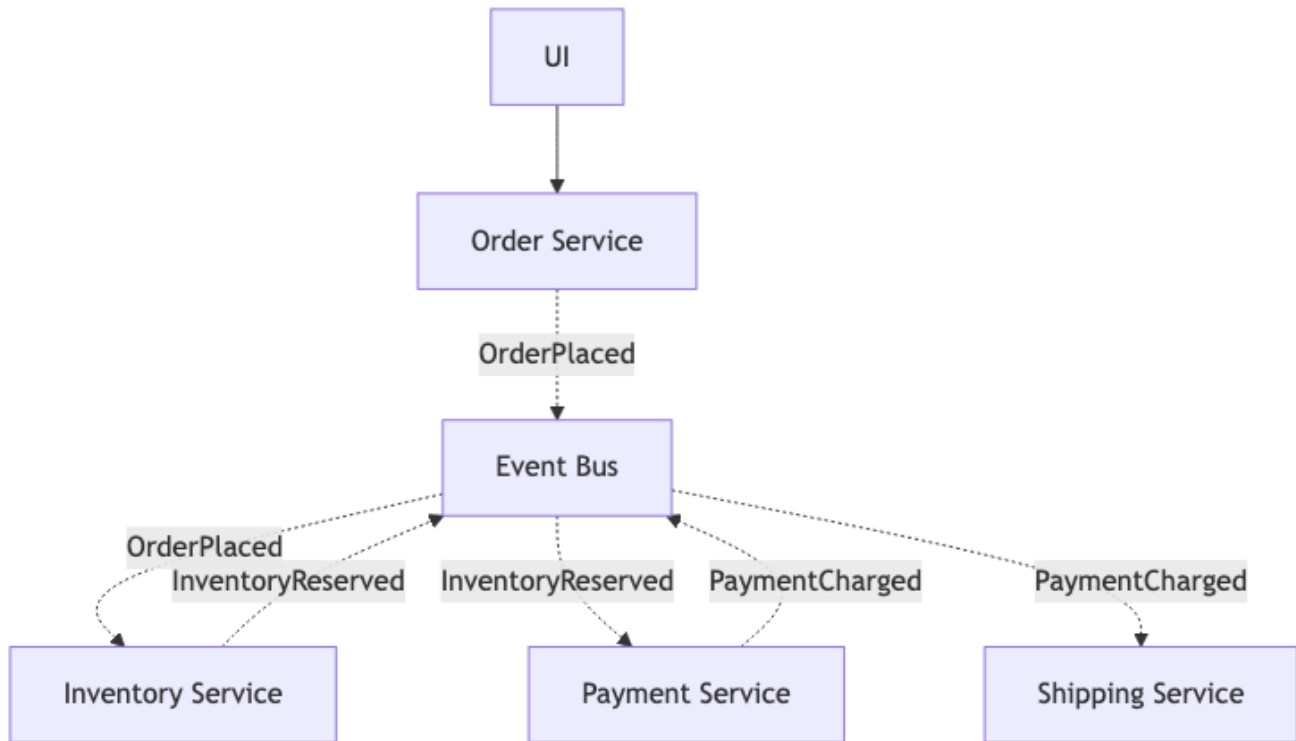
Cons:

- Mediator coupling cao.
- Mediator có thể become god service.

Tools: AWS Step Functions, Camunda, Temporal.

Broker (Choreography)

Không có central điều phối. Service emit event và react event.



Hình 8: Sơ đồ 29

Mỗi service: “khi event X xảy ra, làm Y, emit event Z”.

Pros:

- Decoupling cực mạnh (no mediator).
- Easy add consumer (subscribe event).
- Scalable.

Cons:

- Flow scattered across services. Hard to visualize.
- Compensation phức tạp (saga choreography).
- Debug khó (tracing qua nhiều service).

Tools: Kafka, RabbitMQ, AWS EventBridge, Google Pub/Sub.

Event Bus (Message Broker)

Trung tâm của EDA. Lưu và route messages. Phải:

- **Durable**: không mất message khi broker crash.
- **Ordered** (per partition/queue): events delivered in order.
- **Scalable**: handle millions/giây.
- **Replay-able**: consumer có thể re-process từ point cũ.

Brokers phổ biến:

Broker	Strength	Khi dùng
Apache Kafka	High throughput, replay, partitioning	Stream processing, event sourcing, log aggregation
RabbitMQ	Routing flexibility, simpler ops	Task queue, RPC, low-medium scale
AWS SQS/SNS	Managed, simple	AWS-native, low-medium scale
AWS EventBridge	Schema registry, AWS integration	Cloud-native event bus
Google Pub/Sub	Managed, global	GCP-native, high scale
Redis Streams	In-memory speed, simple	Lightweight, lower durability needs

Kafka là gold standard cho high-scale EDA. Setup phức tạp nhưng capable nhất.

CAP Theorem

Brewer's theorem (2000): trong distributed system có network partition, chỉ chọn 2 trong 3:

- **Consistency**: mọi node thấy data nhất quán.
- **Availability**: mọi request được serve (có thể stale).
- **Partition tolerance**: hệ vẫn chạy khi network split.

Vì network partition không tránh được trong distributed, *phải chọn* giữa C và A.

CP systems

Sacrifice availability cho consistency. Khi partition, prefer return error còn hơn return stale data.

Vd: traditional RDBMS với replication strong consistency, ZooKeeper, etcd.

AP systems

Sacrifice consistency cho availability. Khi partition, return possibly stale data còn hơn 503.

Vd: Cassandra, DynamoDB (eventual consistency mode), most NoSQL.

EDA thường là AP

Event consumers process events khi available. Khi consumer down, events queue lên. Khi consumer back, catch up. Data eventually consistent.

Trade-off: user có thể thấy stale data short term. Business phải accept eventual consistency.

Event design

Event naming

Past tense: OrderPlaced, không PlaceOrder. Event = something happened.

Command (request to do something): PlaceOrder, này là command, không phải event.

Event content

Hai approach:

Thin events (event notification): chỉ ID + key data.

```
{
  "eventType": "OrderPlaced",
  "orderId": "ord_123",
  "timestamp": "2026-01-15T10:30:00Z"
}
```

Consumer cần thêm data □ call API.

Pros: small message, less duplication.

Cons: consumer phải call lại (network round-trip).

Fat events (event-carried state): full data trong event.

```
{
  "eventType": "OrderPlaced",
  "orderId": "ord_123",
  "customerId": "cus_456",
  "items": [{"productId": "p1", "quantity": 2, "price": 100}],
  "total": 200,
  "shippingAddress": {...},
  "timestamp": "2026-01-15T10:30:00Z"
}
```

Consumer không cần call lại.

Pros: self-contained, no extra call.

Cons: large messages, data duplication, schema versioning critical.

Default: fat events cho integration. Thin events cho high-volume internal.

Schema evolution

Events live forever in broker. Schema phải backward compatible.

Approaches:

- **Schema registry** (Confluent Schema Registry, Apicurio): validate schema before publish, enforce compatibility.
- **Versioning**: OrderPlacedV1, OrderPlacedV2. Consumer handle multiple versions hoặc upcast.
- **Avro / Protobuf**: schema-driven serialization với backward/forward compat.

Sai lầm: serialize JSON ad-hoc □ schema drift □ consumer break.

Patterns trong EDA

Event Sourcing

Lưu event log thay vì current state. Reconstruct state bằng replay events.

```
events: [
  AccountOpened(id=1, owner="Alice"),
  Deposited(id=1, amount=100),
  Deposited(id=1, amount=50),
  Withdrew(id=1, amount=30),
]
```

→ Current balance = 100 + 50 - 30 = 120

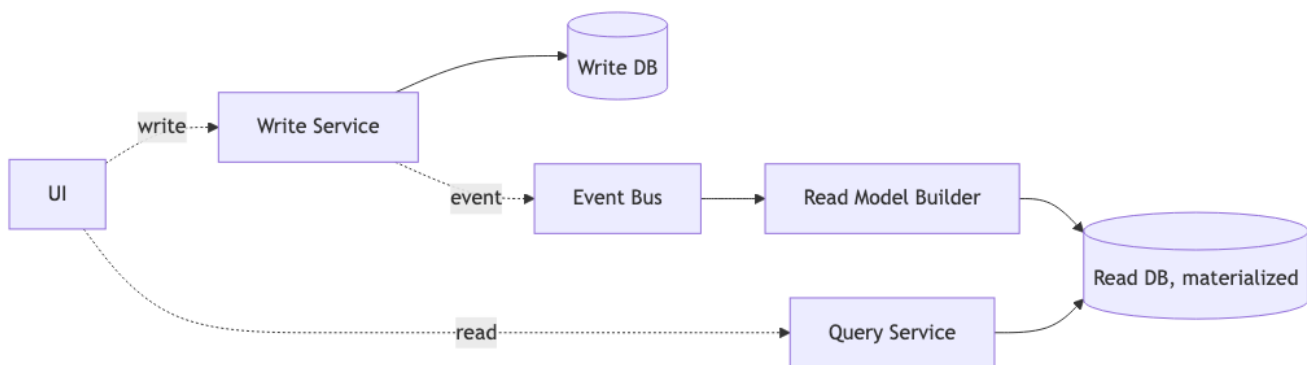
Pros: full audit trail, time-travel debugging, can replay với new logic.

Cons: complexity, query “current state” slow nếu replay mỗi lần.

Mitigate: snapshot periodic.

CQRS (Command Query Responsibility Segregation)

Tách write side (command) khỏi read side (query). Write side emit events; read side materialize views.



Hình 9: Sơ đồ 30

Pros: read/write scale độc lập, optimize read model cho query.

Cons: complexity, eventual consistency between sides.

Use khi: read » write, complex query, multiple view of data.

Saga (đã nhắc Bài 6.3)

Long-running transaction với compensation. Choreography hoặc orchestration.

Trade-off với QA

QA	EDA
Decoupling	□□□□□
Scalability	□□□□□
Throughput	□□□□□
Resilience (consumer down)	□□□□ (queue holds)
Real-time	□□□□
Audit trail	□□□□ (event log)
Eventual consistency	□□

QA	EDA
Debugging	☐☐
Operational complexity	☐ (broker ops)
Sync user query	☐ (not designed for)

EDA tốt cho integration + async workflow + scale. Không thay sync for user-facing query.

Use cases mạnh

- **Microservices integration**: thay sync HTTP between services bằng events.
- **Real-time data pipeline**: stock prices, IoT sensor readings, social feed.
- **Audit logs**: tự nhiên (events là log).
- **Notification system**: user signs up ☐ trigger email + SMS + analytics.
- **Decoupled saga**: orchestrate long workflows.

Anti-patterns

Anti-pattern 1: Event = remote procedure call

Producer emit “GetUser” event, expect single consumer reply. Đó là RPC, không phải event. Dùng HTTP/gRPC.

Event là *notification* về thứ đã xảy ra, không phải request.

Anti-pattern 2: Tightly-coupled events

Producer emit event riêng cho mỗi consumer. Mỗi consumer mới ☐ emit event mới.

Sai. Event nên describe “what happened” (OrderPlaced), không “what should X do” (SendEmailToCustomer).

Anti-pattern 3: No ordering guarantee

Events arrive out of order, consumer break. Vd: OrderShipped before OrderPlaced.

Fix: partition by key (vd: orderId). Within partition, ordering guaranteed.

Anti-pattern 4: No replay capability

Bug in consumer logic. Cần re-process last 24h events. Broker không support ☐ can’t fix.

Fix: dùng Kafka hoặc broker với log-based storage.

Anti-pattern 5: At-least-once delivery, idempotency missing

Broker delivers same event twice (network retry). Consumer side effect twice.

Fix: idempotent consumer. Track processed event IDs. Vd: insert row only if ID not exists.

Tóm tắt

- **EDA**: async event communication, producer-consumer decoupled.
- **Two topologies**: Mediator (orchestration) vs Broker (choreography).
- **Brokers**: Kafka (gold standard), RabbitMQ, cloud-managed.

- **CAP theorem:** EDA thường AP.
- **Event design:** past tense, thin vs fat, schema registry.
- **Patterns:** Event Sourcing, CQRS, Saga.
- **Mạnh cho:** integration, real-time, scale, audit.
- **Tránh:** dùng cho sync query, tightly-coupled events, no ordering, no idempotency.

Bài tiếp: [Space-Based Architecture](#), extreme load architecture.

6.5 Space-Based Architecture

Tóm tắt một dòng: Style đặc thù cho extreme load - mọi data nằm in-memory được replicate qua nodes, DB chỉ sync background. Loại bỏ DB bottleneck nhưng cost cao và niche. Phù hợp Black Friday, ticket sales, real-time auction.

Nếu bạn đến từ Data Science

Space-Based Architecture là niche, nhưng bạn có thể hiểu nó qua online feature serving ở scale rất lớn. Nếu mỗi prediction phải lookup nhiều feature trong vài mili-giây, team có thể đặt feature hot trong memory grid/cache thay vì query database trực tiếp. Application đọc từ memory space, còn database sync phía sau.

Đổi lại, consistency khó hơn. Feature trong memory có thể trễ so với source of truth. Vì vậy style này chỉ hợp khi latency/throughput cực kỳ quan trọng và business chấp nhận eventual consistency.

Vấn đề mà Space-Based giải

Hầu hết web app gặp bottleneck ở database khi load tăng. Even với connection pool, read replica, sharding, DB vẫn là điểm yếu khi load thực sự cao.

Vd: ticket master vài giây trước khi mở bán concert nổi tiếng. 1 triệu user đồng thời. DB connection pool exhausted. App crash.

Space-Based architecture (SBA) giải bằng cách: **loại bỏ DB khỏi hot path**. Mọi data nằm in-memory data grid được replicate qua nhiều processing unit. DB chỉ sync background.

Topology

5 thành phần:

- **Processing Unit (PU):** instance app với in-memory data grid (Hazelcast, Apache Ignite, GigaSpaces). Chứa subset data + business logic.
- **Virtualized Middleware:** cluster manager. Manages PUs, replicates data, routes requests.
- **Data Reader:** async load data từ DB vào grid khi PU start.
- **Data Writer:** async write changes từ grid $\rightarrow$ DB.
- **Data pump:** async stream của events từ grid $\rightarrow$ DB.

Tên gọi “space-based”

“Space” = “tuple space”, distributed shared memory concept từ 1980s (Linda language). PUs share một “space” of data qua replication. Mọi PU thấy cùng data như nếu chúng truy cập shared memory.

Cách hoạt động

Đọc

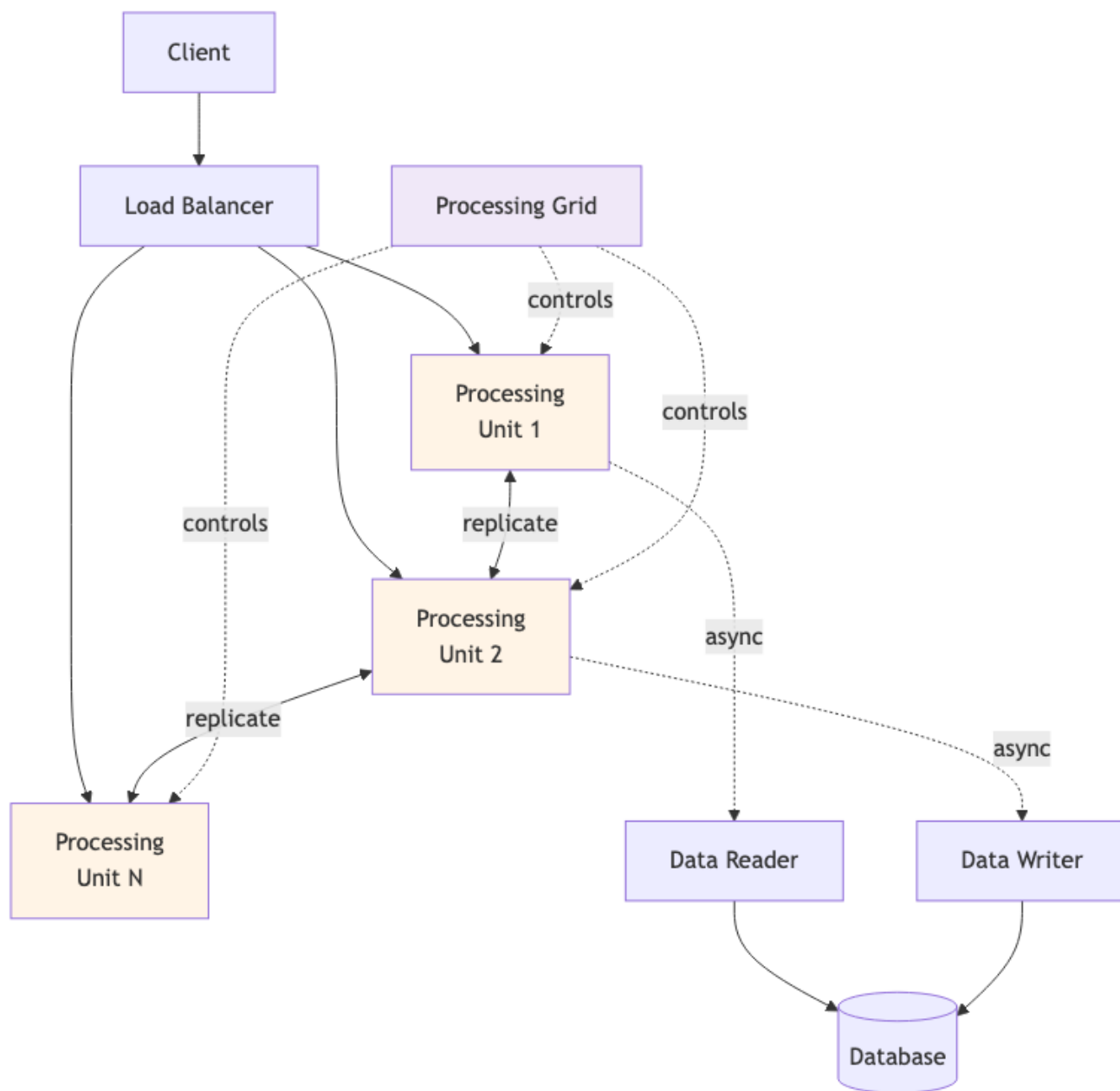
User request $\rightarrow$ LB route đến PU $\rightarrow$ PU đọc data từ grid in-memory (nanoseconds). Không touch DB.

Ghi

User request $\rightarrow$ LB route $\rightarrow$ PU update grid in-memory $\rightarrow$ grid replicate update đến PUs khác (milliseconds) $\rightarrow$ background, Data Writer flush update đến DB (seconds-minutes).

Scale

Thêm PU $\rightarrow$ joins grid $\rightarrow$ automatically gets data subset từ existing PUs. Linear scaling.



Hình 10: Sơ đồ 31

Failover

PU crash → grid detect → data trên crashed PU đã replicate ở PUs khác → no data loss. New PU spin up → join → re-balance.

Khi dùng

→ Phù hợp:

- **Extreme load spike:** Black Friday, ticket sales, flash sale.
- **Real-time auction/trading:** stock market, betting, ad bidding.
- **Online gaming:** massively multiplayer real-time.
- **Streaming analytics:** real-time aggregation.

Common: load không predictable, scaling cần instant, latency phải sub-100ms.

→ Không phù hợp:

- App load steady (use service-based hoặc microservices).
- App với complex query joins (in-memory grid không tốt cho ad-hoc query).
- App với strict consistency requirements (in-memory grid là eventual).
- Team không có experience với Hazelcast/Ignite/GigaSpaces.

Trade-off

QA	Space-Based
Throughput (extreme load)	□□□□□
Latency (read)	□□□□□ (nanoseconds)
Scalability (elastic)	□□□□□
Availability	□□□□
Consistency	□□ (eventual)
Operational complexity	□
Cost	□ (expensive infrastructure)
Suitable workloads	Narrow
Learning curve	□

Niche style. 90%+ projects không cần.

Vendors

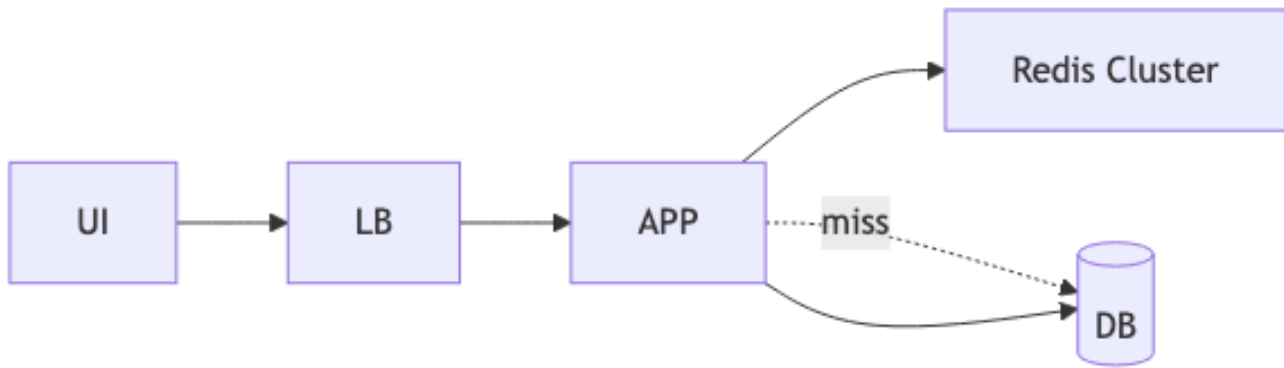
- **Hazelcast IMDG** (open source + enterprise).
- **Apache Ignite** (open source).
- **GigaSpaces** (commercial, pioneered SBA term).
- **Oracle Coherence** (enterprise).
- **Redis Enterprise** (gần SBA, in-memory grid).

Setup phức tạp. Cần team với deep distributed systems experience.

Variant: Distributed Cache + DB

Variant đơn giản hơn full SBA: dùng Redis cluster làm cache layer trước DB. Read từ cache, fallback DB. Write cache + DB.

Lợi ích phần lớn của SBA (read latency) mà ít complexity hơn.



Hình 11: Sơ đồ 32

Đa số “we need SBA” thực sự “we need distributed cache + sharding”. Cân nhắc trước khi đi full SBA.

Sai lầm thường gặp

Sai lầm 1: Choose SBA cho load bình thường

App 10k req/s □ service-based đủ. SBA overkill. Cost 10x mà không có lợi ích.

Sai lầm 2: Quên data persistence

Toàn bộ data in-memory. Power loss □ mất hết nếu không sync to DB.

Fix: data pump sync regularly. Snapshot periodic. Persistent grid (Ignite) với disk backing.

Sai lầm 3: Strong consistency expectation

Business team expect “100% accurate balance always”. SBA = eventual consistency. Mismatch.

Fix: educate stakeholder; consider non-SBA alternative if requirement strict.

Sai lầm 4: Complex query

In-memory grid tốt cho key-value lookup. Complex JOIN, aggregation: slow.

Fix: pre-compute aggregations. Push complex queries to read replica DB asynchronously.

Tóm tắt

- **Space-Based**: in-memory grid + async DB sync.
- **Loại bỏ DB khỏi hot path** □ unlimited scalability.
- **Use case niche**: extreme load spikes, real-time auction/gaming.
- **Cost cao**: infrastructure, expertise, complexity.
- **Alternative simpler**: distributed cache (Redis) + sharded DB.

Tổng kết Phần 6

4 distributed styles:

Style	Best for	Complexity
Service-based	Enterprise medium, default distributed	Medium
Microservices	Large team + complex domain	High
Event-Driven	Async integration, real-time	Medium-High
Space-Based	Extreme load, niche	Very High

Default: Service-based. Overlay Event-Driven cho async parts. Microservices khi thực sự cần. Space-based khi extreme.

Phần tiếp: [Phần 7 - Documenting Architecture](#), kiến trúc không document là kiến trúc chết.